

Introduction to C# App Programming

Chapter 3 of Visual C# How to Program, 6/e

OBJECTIVES

In this chapter you'll:

- Write simple C# apps using code rather than visual programming.
- Input data from the keyboard and output data to the screen.
- Use C# 6's `string` interpolation to create formatted `strings` by inserting values into `string` literals.
- Declare and use data of various types.
- Store data in memory and retrieve it.
- Use arithmetic operators.
- Determine the order in which operators are applied.
- Write decision-making statements with equality and relational operators.

3.1 Introduction

3.2 Simple App: Displaying a Line of Text

- 3.2.1 Comments
- 3.2.2 **using** Directive
- 3.2.3 Blank Lines and Whitespace
- 3.2.4 Class Declaration
- 3.2.5 **Main** Method
- 3.2.6 Displaying a Line of Text
- 3.2.7 Matching Left ({) and Right (}) Braces

3.3 Creating a Simple App in Visual Studio

- 3.3.1 Creating the Console App
- 3.3.2 Changing the Name of the App File
- 3.3.3 Writing Code and Using *IntelliSense*
- 3.3.4 Compiling and Running the App
- 3.3.5 Syntax Errors, Error Messages and the **Error List** Window

3.4 Modifying Your Simple C# App

- 3.4.1 Displaying a Single Line of Text with Multiple Statements
- 3.4.2 Displaying Multiple Lines of Text with a Single Statement

3.5 String Interpolation

3.6 Another C# App: Adding Integers

- 3.6.1 Declaring the `int` Variable `number1`
- 3.6.2 Declaring Variables `number2` and `sum`
- 3.6.3 Prompting the User for Input
- 3.6.4 Reading a Value into Variable `number1`
- 3.6.5 Prompting the User for Input and Reading a Value into `number2`
- 3.6.6 Summing `number1` and `number2`
- 3.6.7 Displaying the `sum` with `string` Interpolation
- 3.6.8 Performing Calculations in Output Statements

3.7 Memory Concepts

3.8 Arithmetic

- 3.8.1 Arithmetic Expressions in Straight-Line Form
- 3.8.2 Parentheses for Grouping Subexpressions
- 3.8.3 Rules of Operator Precedence
- 3.8.4 Sample Algebraic and C# Expressions
- 3.8.5 Redundant Parentheses

3.9 Decision Making: Equality and Relational Operators

3.10 Wrap-Up

3.1 Introduction

- ▶ **Console apps** input and output text in a **console window**, which in Windows is known as the **Command Prompt**.
- ▶ Figure 3.1 shows the app's source code and output.

3.2 Simple App: Displaying a Line of Text

- ▶ Figure 3.1 shows the app's source code and output.

```
1  // Fig. 3.1: Welcome1.cs
2  // Text-displaying app.
3  using System;
4
5  class Welcome1
6  {
7      // Main method begins execution of C# app
8      static void Main()
9      {
10         Console.WriteLine("Welcome to C# Programming!");
11     } // end Main
12 } // end class Welcome1
```

Welcome to C# Programming!

Fig. 3.1 | Text-displaying app.

3.2.1 Comments

- ▶ You'll insert **comments** to document your apps and improve their readability.
- ▶ The C# compiler ignores comments, so they do *not* cause the computer to perform any action when the app executes.
- ▶ A comment that begins with `//` is called a **single-line comment**, because it terminates at the end of the line on which it appears.

3.2.1 Comments

- ▶ A `//` comment also can begin in the middle of a line and continue until the end of that line.
- ▶ **Delimited comments** begin with the delimiter `/*` and end with the delimiter `*/`. All text between the delimiters is ignored by the compiler.



Common Programming Error 3.1

*Forgetting one of the delimiters of a delimited comment is a syntax error. A programming language's **syntax** specifies the grammatical rules for writing code in that language. A **syntax error** occurs when the compiler encounters code that violates C#'s language rules. In this case, the compiler does not produce an executable file. Instead, it issues one or more error messages to help you identify and fix the incorrect code. Syntax errors are also called **compiler errors**, **compile-time errors** or **compilation errors**, because the compiler detects them during the compilation phase. You cannot execute your app until you correct all the compilation errors in it. We'll see that some compile-time errors are not syntax errors.*



Error-Prevention Tip 3.1

When the compiler reports an error, the error may not be in the line indicated by the error message. First, check the line for which the error was reported. If that line does not contain syntax errors, check several preceding lines.

3.2.2 using Directive

- ▶ A **using directive** tells the compiler where to look for a predefined class that's used in an app.
- ▶ Predefined classes are organized under **namespaces**—named collections of related classes. Collectively, .NET's predefined namespaces are known as the **.NET Framework Class Library**.
- ▶ The `System` namespace contains the predefined `Console` class and many other useful classes.



Error-Prevention Tip 3.2

Forgetting to include a `using` directive for a namespace that contains a class used in your app typically results in a compilation error, containing a message such as “The name 'Console' does not exist in the current context.” When this occurs, check that you provided the proper `using` directives and that the names in them are spelled correctly, including proper use of uppercase and lowercase letters. In the editor, when you hover over an error’s red squiggly line, Visual Studio displays a box containing the link “[Show potential fixes](#).” If a `using` directive is missing, one potential fix shown is to add the `using` directive to your code—simply click that fix to have Visual Studio edit your code.

3.2.3 Blank Lines and Whitespace

- ▶ Blank lines and space characters make code easier to read and together with tab characters are known as **whitespace**.
- ▶ Whitespace is ignored by the compiler.

3.2.4 Class Declaration

- ▶ **Keywords** (sometimes called **reserved words**) are reserved for use by C# and are always spelled with all lowercase letters.
- ▶ Every app consists of at least one **class declaration** that is defined by the programmer. These are known as **user-defined classes**.
- ▶ The **class** keyword introduces a class declaration and is immediately followed by the **class name**.

3.2.4 Class Declaration

- ▶ A class name is an **identifier**:
 - Series of characters consisting of letters, digits and underscores.
 - Cannot begin with a digit and does not contain spaces.
- ▶ The complete list of C# keywords is shown in Fig. 3.2.
- ▶ The contextual keywords in Fig. 3.2 can be used as identifiers outside the contexts in which they are keywords, but for clarity this is not recommended.



Good Programming Practice 3.1

By convention, always begin a class name's identifier with a capital letter and start each subsequent word in the identifier with a capital letter.



Common Programming Error 3.2

C# is case sensitive. Not using the proper uppercase and lowercase letters for an identifier normally causes a compilation error.



Common Programming Error 3.3

Using a keyword as an identifier is a compilation error.

Keywords and contextual keywords

<code>abstract</code>	<code>as</code>	<code>base</code>	<code>bool</code>	<code>break</code>	<code>byte</code>
<code>case</code>	<code>catch</code>	<code>char</code>	<code>checked</code>	<code>class</code>	<code>const</code>
<code>continue</code>	<code>decimal</code>	<code>default</code>	<code>delegate</code>	<code>do</code>	<code>double</code>
<code>else</code>	<code>enum</code>	<code>event</code>	<code>explicit</code>	<code>extern</code>	<code>false</code>
<code>finally</code>	<code>fixed</code>	<code>float</code>	<code>for</code>	<code>foreach</code>	<code>goto</code>
<code>if</code>	<code>implicit</code>	<code>in</code>	<code>int</code>	<code>interface</code>	<code>internal</code>
<code>is</code>	<code>lock</code>	<code>long</code>	<code>namespace</code>	<code>new</code>	<code>null</code>
<code>object</code>	<code>operator</code>	<code>out</code>	<code>override</code>	<code>params</code>	<code>private</code>
<code>protected</code>	<code>public</code>	<code>readonly</code>	<code>ref</code>	<code>return</code>	<code>sbyte</code>
<code>sealed</code>	<code>short</code>	<code>sizeof</code>	<code>stackalloc</code>	<code>static</code>	<code>string</code>
<code>struct</code>	<code>switch</code>	<code>this</code>	<code>throw</code>	<code>true</code>	<code>try</code>
<code>typeof</code>	<code>uint</code>	<code>ulong</code>	<code>unchecked</code>	<code>unsafe</code>	<code>ushort</code>
<code>using</code>	<code>virtual</code>	<code>void</code>	<code>volatile</code>	<code>while</code>	

Fig. 3.2 | Keywords and contextual keywords. (Part 1 of 2.)

Keywords and contextual keywords

Contextual Keywords

add	alias	ascending	async	await	by
descending	dynamic	equals	from	get	global
group	into	join	let	nameof	on
orderby	partial	remove	select	set	value
var	where	yield			

Fig. 3.2 | Keywords and contextual keywords. (Part 2 of 2.)

3.2.4 Class Declaration

- ▶ C# is **case sensitive**—that is, uppercase and lowercase letters are distinct, so `a1` and `A1` are different (but both valid) identifiers.



Good Programming Practice 3.2

By convention, a file that contains a single class should have a name that's identical to the class name (plus the .CS extension) in both spelling and capitalization. This makes it easy to identify which file contains the class's declaration.

3.2.4 Class Declaration

- ▶ A **left brace**, {, begins the **body** of every class declaration.
- ▶ A corresponding **right brace**, }, must end each class declaration.



Good Programming Practice 3.3

Indent the entire body of each class declaration one “level” of indentation between the left and right braces that delimit the body of the class. This format emphasizes the class declaration’s structure and makes it easier to read. You can let the IDE format your code by selecting Edit > Advanced > Format Document.



Good Programming Practice 3.4

*Set a convention for the indent size you prefer, then uniformly apply that convention. The **Tab** key may be used to create indents, but tab stops vary among text editors. Microsoft recommends four-space indents, which is the default in Visual Studio. Due to the limited width of code lines in print books, we use three-space indents—this reduces the number of code lines that wrap to a new line, making the code a bit easier to read. We show how to set the tab size in the Before You Begin section that follows the Preface.*



Error-Prevention Tip 3.3

Whenever you type an opening left brace, {, in your app, the IDE immediately inserts the closing right brace, }, then repositions the cursor between the braces so you can begin typing the body. This practice helps prevent errors due to missing braces.



Common Programming Error 3.4

It's a syntax error if braces do not occur in matching pairs.

3.2.5 Main Method

- ▶ **Parentheses** after an identifier indicate that it is an app building block called a method. Class declarations normally contain one or more methods.
- ▶ Method names usually follow the same casing capitalization conventions used for class names.
- ▶ For each app, one of the methods in a class must be called `Main`; otherwise, the app will not execute.
- ▶ A method performs a task and returns information it completes its task.
- ▶ Keyword **void** indicates that `Main` will not return any information.
- ▶ The **body** of a method declaration begins with a left brace and ends with a corresponding right brace.



Good Programming Practice 3.5

Indent each method declaration's body statements one level of indentation between the left and right braces that define the body.

3.2.6 Displaying a Line of Text

- ▶ Characters between double quotation marks are **strings**.
- ▶ Whitespace characters in strings are *not* ignored by the compiler.
- ▶ The **Console.WriteLine method** displays a line of text in the console window.
- ▶ The string in parentheses is the **argument** to the **Console.WriteLine method**.
- ▶ Method **Console.WriteLine** performs its task by displaying (also called outputting) its argument in the console window.

3.2.6 Displaying a Line of Text

- ▶ A method is typically composed of one or more **statements** that perform the method's task.
- ▶ Most statements end with a semicolon.

3.3.1 Creating the Console App

- ▶ Select **File > New > Project...** to display the **New Project** dialog (Fig. 3.3).
- ▶ At the left side of the dialog, under **Installed > Templates > Visual C#** select the **Windows** category, then in the middle of the dialog select the **Console Application** template.
- ▶ In the dialog's **Name** field, type `Welcome1`, then click **OK** to create the project.

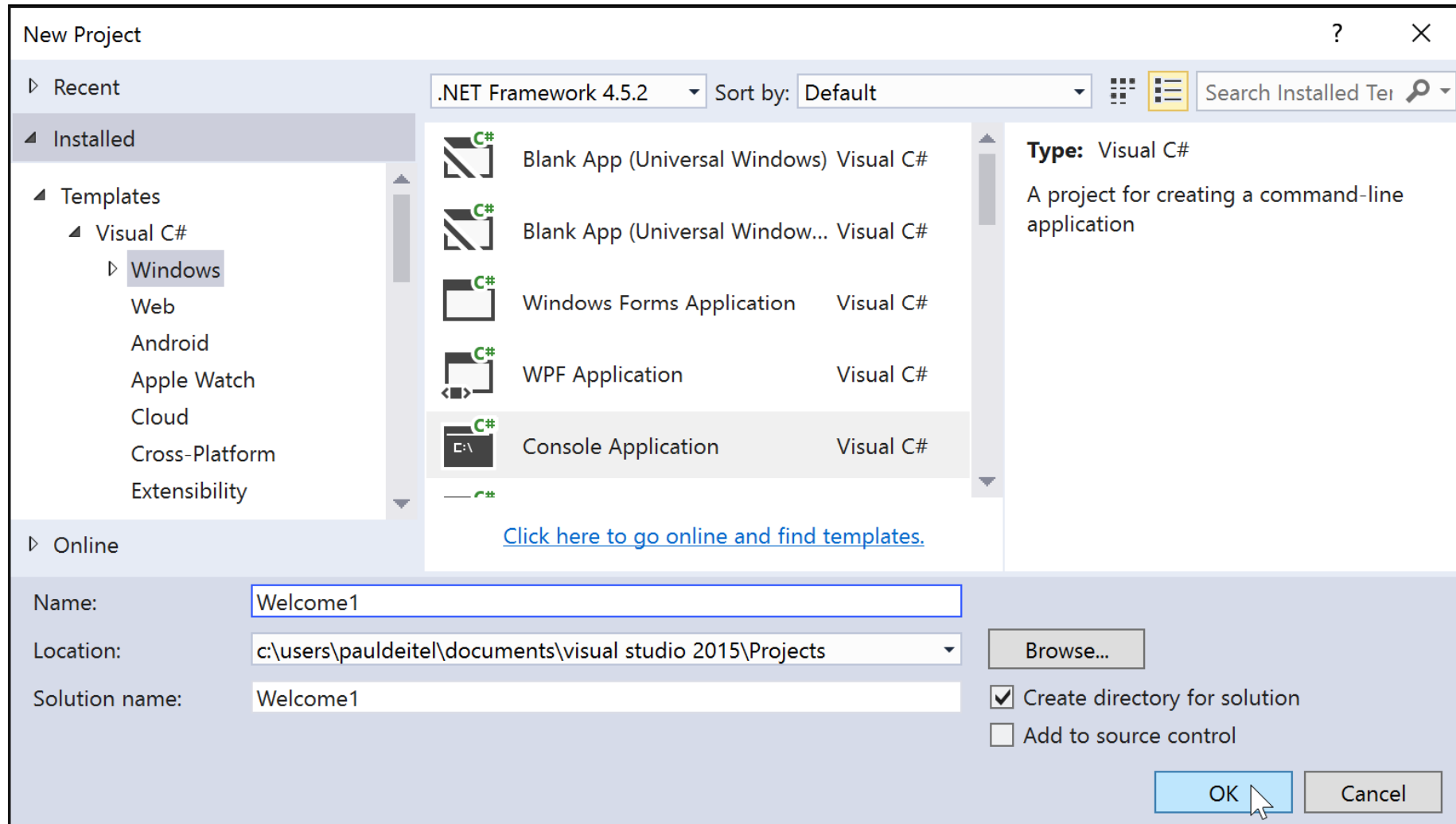


Fig. 3.3 | Selecting **Console Application** in the **New Project** dialog.

3.3 Creating a Simple App in Visual Studio (Cont.)

- ▶ The IDE now contains the open console app.
- ▶ The code coloring scheme used by the IDE is called **syntax-color highlighting** and helps you visually differentiate code elements.

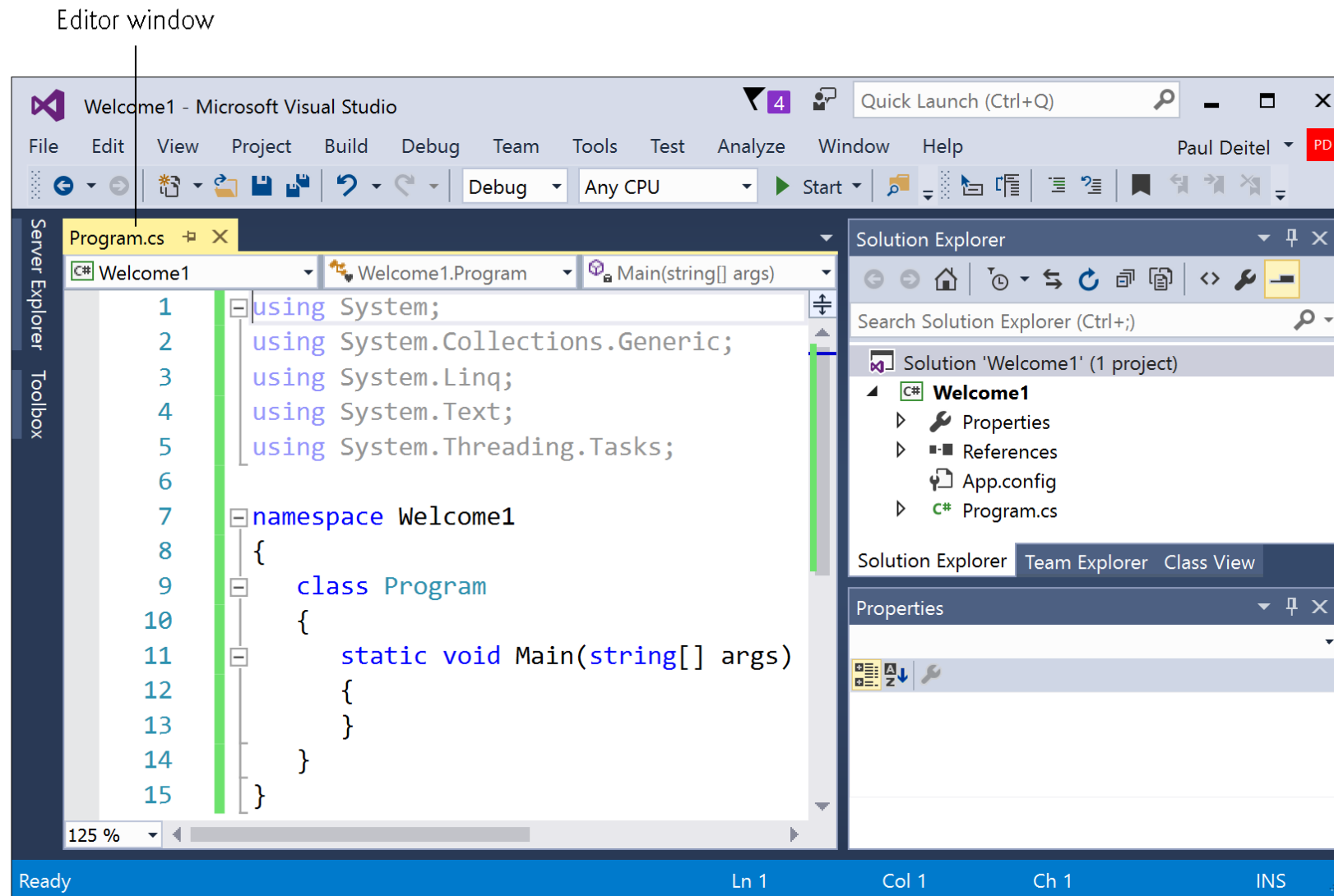


Fig. 3.4 | IDE with an open console app's code displayed in the editor and the project's contents shown in the **Solution Explorer** at the IDE's top-right side.

3.3.2 Changing the Name of the App File

- ▶ To rename the app file, right click `Program.cs` in the **Solution Explorer** window and select **Rename** window
- ▶ Type `Welcome1` then press *Enter*.



Error-Prevention Tip 3.4

When changing a file name in a Visual Studio project, always do so in Visual Studio. Changing file names outside the IDE can break the project and prevent it from executing.

3.3.3 Writing Code and Using *IntelliSense*

- ▶ *IntelliSense* lists a class's **members**, which include method names.
- ▶ As you type, *IntelliSense* lists various items that start with or contain the letters you've typed so far.
- ▶ *IntelliSense* also displays a tool tip containing a description of the first matching item.

3.3.3 Writing Code and Using *IntelliSense*

- ▶ You can either type the complete member name, double click the member name in the member list or press the *Tab* key to complete the name.
- ▶ While the *IntelliSense* window is displayed pressing the *Ctrl* key makes the window transparent so you can see the code behind the window.

a) *IntelliSense* window displayed as you type

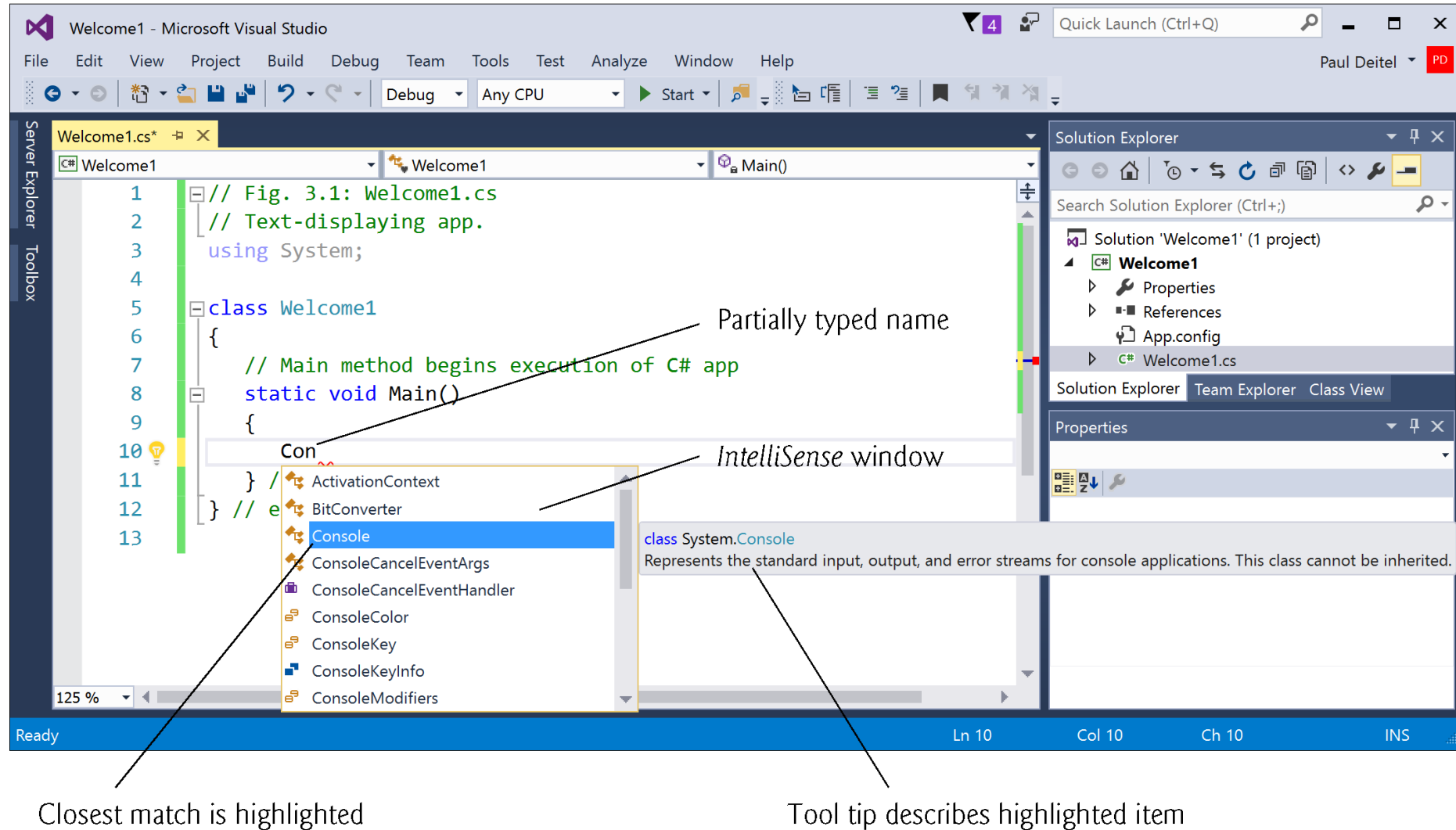
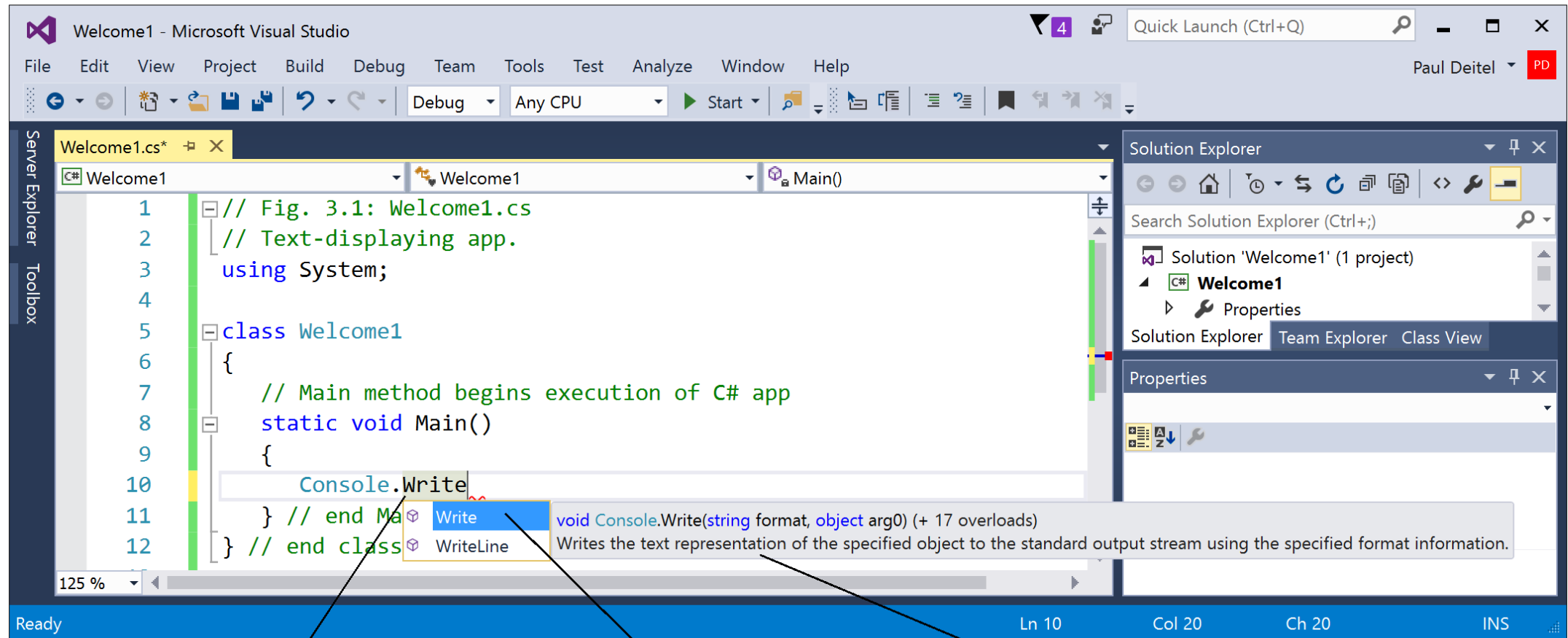


Fig. 3.5 | *IntelliSense* window as you type “Con”.

b) *IntelliSense* window showing method names that start with **Write**



Partially typed member

Highlighted member

Tool tip describes highlighted member

Fig. 3.6 | *IntelliSense* window.

3.3.3 Writing Code and Using *IntelliSense*

- ▶ When you type the open parenthesis character, (, after a method name, the *Parameter Info* window is displayed (Fig. 3.7).
- ▶ This window contains information about the method's parameters.
- ▶ Up and down arrows allow you to scroll through overloaded versions of the method.

Parameter Info window

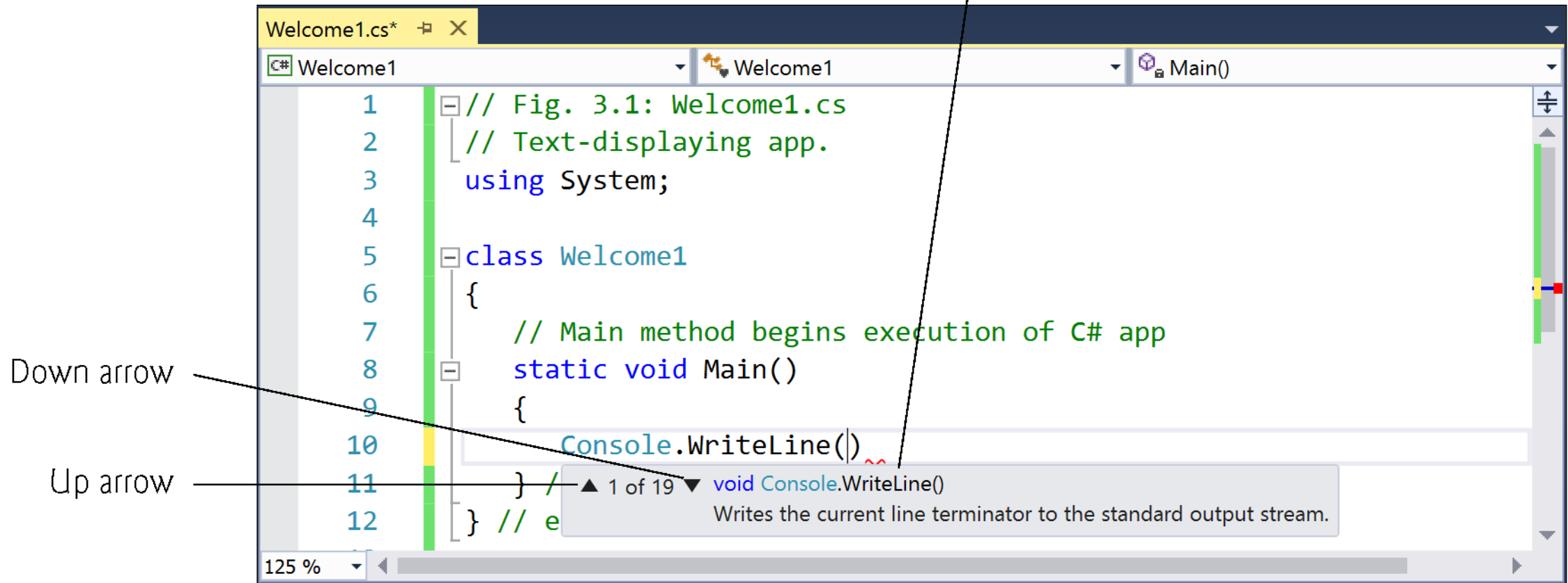


Fig. 3.7 | *Parameter Info window.*

3.3.4 Compiling and Running the App

- ▶ To compile the app, select **Build > Build Solution**.
 - This invokes the `Main` method.
- ▶ Figure 3.8 shows the results of executing this app, displayed in a console (**Command Prompt**) window.



Fig. 3.8 | Executing the app shown in Fig. 3.1.

3.3.5 Errors, Error Messages and the Error List Window

- ▶ As you type code, the IDE responds either by applying syntax-color highlighting or by generating a **syntax error**.
- ▶ A syntax error indicates a violation of Visual C#'s rules for creating correct apps.
- ▶ When a syntax error occurs, the IDE underlines the location of the error with a red squiggly line and provides a description of it in the **Error List window** (Fig. 3.9).



Error-Prevention Tip 3.5

One compile-time error can lead to multiple entries in the Error List window. Each error you correct could eliminate several subsequent error messages when you recompile your app. So when you see an error you know how to fix, correct it—the IDE will recompile your code in the background, so fixing an error may make several other errors disappear.

Intentionally omitted semicolon (syntax error)

The screenshot shows a Visual Studio IDE with a C# file named `Welcome1.cs`. The code is as follows:

```
1 // Fig. 3.1: Welcome1.cs
2 // Text-displaying app.
3 using System;
4
5 class Welcome1
6 {
7     // Main method begins execution of C# app
8     static void Main()
9     {
10         Console.WriteLine("Welcome to C# Programming!")
11     } // end Main
12 } // end class Welcome1
```

Line 10 has a squiggly red underline under the closing parenthesis of `Console.WriteLine`, indicating a syntax error. The `Error List` window at the bottom shows the following error:

	Code	Description	Project	File	Line	Suppression S...
✖	CS1002	; expected	Welcome1	Welcome1.cs	10	Active

Labels with arrows point to the `Error List` window, the error description `; expected`, and the squiggly underline on line 10.

Error List window

Error description(s)

Squiggly underline indicates a syntax error

3.4.1 Displaying a Single Line of Text with Multiple Statements

- Class `Welcome2`, shown in Fig. 3.10, uses two statements to produce the same output as that shown in the previous example.
- Unlike `WriteLine`, the `Console` class's `Write` method does not position the screen cursor at the beginning of the next line in the console window.

```
1 // Fig. 3.10: Welcome2.cs
2 // Displaying one line of text with multiple statements.
3 using System;
4
5 class Welcome2
6 {
7     // Main method begins execution of C# app
8     static void Main()
9     {
10         Console.Write("Welcome to ");
11         Console.WriteLine("C# Programming!");
12     } // end Main
13 } // end class Welcome2
```

Welcome to C# Programming!

Fig. 3.10 | Displaying one line of text with multiple statements.

3.4.2 Displaying Multiple Lines of Text with a Single Statement

- A single statement can display multiple lines by using newline characters.
- Like space characters and tab characters, newline characters are whitespace characters.
- The app of Fig. 3.11 outputs four lines of text, using newline characters to indicate when to begin each new line.

```
1  // Fig. 3.11: Welcome3.cs
2  // Displaying multiple lines with a single statement.
3  using System;
4
5  class Welcome3
6  {
7      // Main method begins execution of C# app
8      static void Main()
9      {
10         Console.WriteLine("Welcome\n\tto\n\tC#\n\tProgramming!");
11     } // end Main
12 } // end class Welcome3
```

```
Welcome
to
C#
Programming!
```

Fig. 3.11 | Displaying multiple lines with a single statement.

3.4.2 Displaying Multiple Lines of Text with a Single Statement

- ▶ The **backslash** (\) is called an **escape character**, and is used as the first character in an **escape sequence**.
- ▶ The escape sequence `\n` represents the **newline character**.
- ▶ Figure 3.12 lists several common escape sequences and describes how they affect the display of characters in the console window.

Escape sequence	Description
<code>\n</code>	Newline. Positions the screen cursor at the beginning of the next line.
<code>\t</code>	Horizontal tab. Moves the screen cursor to the next tab stop.
<code>\"</code>	Double quote. Used to place a double-quote character (") in a string—e.g., <code>Console.WriteLine("\\"in quotes\\");</code> displays "in quotes".
<code>\r</code>	Carriage return. Positions the screen cursor at the beginning of the current line—does not advance the cursor to the next line. Any characters output after the carriage return overwrite the characters previously output on that line.
<code>\\</code>	Backslash. Used to place a backslash character in a string.

Fig. 3.12 | Common escape sequences.

3.5 String Interpolation

- Many programs format data into strings.
- C# 6 introduces a mechanism called **string interpolation** that enables you to insert values in string literals to create formatted strings.
- Figure 3.13 demonstrates this capability.


```
1 // Fig. 3.13: Welcome4.cs
2 // Inserting content into a string with string interpolation.
3 using System;
4
5 class Welcome4
6 {
7     // Main method begins execution of C# app
8     static void Main()
9     {
10         string person = "Paul"; // variable that stores the string "Paul"
11         Console.WriteLine($"Welcome to C# Programming, {person}!");
12     } // end Main
13 } // end class Welcome4
```

Welcome to C# Programming, Paul!

Fig. 3.13 | Inserting content into a string with string interpolation.

3.5 String Interpolation

- ▶ A **variable declaration statement** (also called a **declaration**) specifies the name and type of a variable.
- ▶ A variable is a location in the computer's memory where a value can be stored for later use.
- ▶ Variables are declared with a name and a type before they're used:
 - ▶ A variable's name enables the app to access the corresponding value in
 - ▶ A variable's type specifies what kind of information is stored at that location in memory.
- ▶ Variables of type string store character-based information.
- ▶ Like other statements, declaration statements end with a semicolon (;).

3.5 String Interpolation

- ▶ string interpolation inserts values into a `string`.
- ▶ An interpolated `string` must begin with a `$` (dollar sign). Then, you can insert interpolation expressions enclosed in braces, `{ }`, anywhere between the quotes (`"`).
- ▶ When `C#` encounters an interpolated `string`, it replaces each braced interpolation expression with the corresponding value.

3.6 Another C# App: Adding Integers

- Applications remember numbers and other data in the computer's memory and access that data through app elements called **variables**.
- A **variable** is a location in the computer's memory where a value can be stored for use later in an app.
- A **variable declaration statement** (also called a **declaration**) specifies the name and type of a variable.
 - A variable's name enables the app to access the value of the variable in memory—the name can be any valid identifier.
 - A variable's type specifies what kind of information is stored at that location in memory.

```
1  // Fig. 3.14: Addition.cs
2  // Displaying the sum of two numbers input from the keyboard.
3  using System;
4
5  class Addition
6  {
7      // Main method begins execution of C# app
8      static void Main()
9      {
10         int number1; // declare first number to add
11         int number2; // declare second number to add
12         int sum; // declare sum of number1 and number2
13
14         Console.Write("Enter first integer: "); // prompt user
15         // read first number from user
16         number1 = int.Parse(Console.ReadLine());
```

Fig. 3.14 | Displaying the sum of two numbers input from the keyboard. (Part 1 of 2.)

```
17
18     Console.Write("Enter second integer: "); // prompt user
19     // read second number from user
20     number2 = int.Parse(Console.ReadLine());
21
22     sum = number1 + number2; // add numbers
23
24     Console.WriteLine($"Sum is {sum}"); // display sum
25 } // end Main
26 } // end class Addition
```

```
Enter first integer: 45
Enter second integer: 72
Sum is 117
```

Fig. 3.14 | Displaying the sum of two numbers input from the keyboard. (Part 2 of 2.)

3.6.1 Declaring the `int` Variable `number1`

- Variables of type `int` store `integer` values (whole numbers such as 7, -11, 0 and 31914).
- Types `float`, `double` and `decimal` specify real numbers (numbers with decimal points).
- Type `char` represents a single character.
- These types are called `simple types`. Simple-type names are keywords and must appear in all lowercase letters.

3.6.2 Declaring Variables `number2` and `sum`



Good Programming Practice 3.6

Declare each variable on a separate line. This format allows a comment to be easily inserted next to each declaration.



Good Programming Practice 3.7

*Choosing meaningful variable names helps code to be **self-documenting** (i.e., one can understand the code simply by reading it rather than by reading documentation manuals or viewing an excessive number of comments).*



Good Programming Practice 3.8

By convention, variable-name identifiers begin with a lowercase letter, and every word in the name after the first word begins with a capital letter (e.g., `firstNumber`). This naming convention is known as camel case.

3.6.3 Prompting the User for Input

- A **prompt** because it directs the user to take a specific action

3.6.4 Reading a Value into Variable number1

- Console's **ReadLine** method waits for the user to type a string of characters at the keyboard and press the *Enter* key.
- **ReadLine** returns the text the user entered.
- The **int.Parse** method converts this sequence of characters into data of type **int**.

3.6.4 Reading a Value into Variable number1

- A value can be stored in a variable using the **assignment operator**, **=**.
- Operator **=** is called a **binary operator**, because it works on two pieces of information, or **operands**.
- An **assignment statement** assigns a value to a variable.
- Everything to the right of the assignment operator, **=**, is always evaluated before the assignment is performed.



Good Programming Practice 3.9

Place spaces on either side of a binary operator to make the code more readable.

3.6.5 Summing `number1` and `number2`

- An **expression** is any portion of a statement that has a value associated with it.
 - The value of the expression `number1 + number2` is the sum of the numbers.
 - The value of the expression `Console.ReadLine()` is the string of characters typed by the user.
- Calculations can also be performed inside output statements.

3.7 Memory Concepts

- Variable names actually correspond to **locations** in the computer's memory.
- Every variable has a **name**, a **type**, a **size** and a **value**.
- In Fig. 3.15, the computer has placed the value 45 in the memory location corresponding to number1.

number1



45

Fig. 3.15 | Memory location showing the name and value of variable `number1`.

3.7 Memory Concepts (Cont.)

- ▶ In Fig. 3.16, 72 has been placed in location number 2.

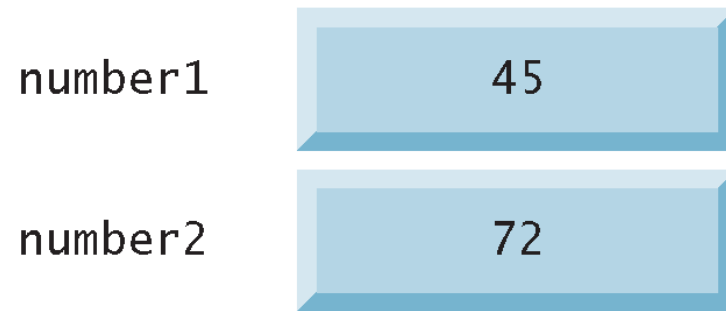


Fig. 3.16 | Memory locations after storing values for number1 and number2.

3.7 Memory Concepts (Cont.)

- After Sum has been calculated, memory appears as shown in Fig. 3.17.

number1	45
number2	72
sum	117

Fig. 3.17 | Memory locations after calculating and storing the sum of `number1` and `number2`.

3.7 Memory Concepts (Cont.)

- Whenever a value is placed in a memory location, the value replaces the previous value in that location, and the previous value is lost.
- When a value is read from a memory location, the process is nondestructive.

3.8 Arithmetic

- ▶ The **arithmetic operators** are summarized in Fig. 3.18.
- ▶ The arithmetic operators in Fig. 3.18 are binary operators.

C# operation	Arithmetic operator	Algebraic expression	C# expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	$b \cdot m$	<code>b * m</code>
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>
Remainder	%	$r \bmod s$	<code>r % s</code>

Fig. 3.18 | Arithmetic operators.

3.8 Arithmetic (Cont.)

- **Integer division** yields an integer quotient—any fractional part in integer division is simply *truncated* (discarded)—*no rounding* occurs.
- C# provides the remainder operator, %, which yields the remainder after division.
- The remainder operator is most commonly used with integer operands but can also be used with floats, doubles, and decimals.

3.8 Arithmetic (Cont.)

- Arithmetic expressions must be written in **straight-line form** to facilitate entering an app's code into the computer.
 - Expressions such as “a divided by b” must be written as a / b in a straight line.
- Parentheses are used to group terms in C# expressions in the same manner as in algebraic expressions.
- If an expression contains **nested parentheses**, the expression in the innermost set of parentheses is evaluated first.

3.8 Arithmetic (Cont.)

- ▶ Arithmetic operators are evaluated according to the **rules of operator precedence**, which are generally the same as those followed in algebra (Fig. 3.19).
- ▶ As in algebra, it is acceptable to place unnecessary (**redundant**) parentheses in an expression to make the expression clearer.

Operators	Operations	Order of evaluation (associativity)
<i>Evaluated first</i>		
*	Multiplication	If there are several operators of this type, they're evaluated from left to right.
/	Division	
%	Remainder	
<i>Evaluated next</i>		
+	Addition	If there are several operators of this type, they're evaluated from left to right.
-	Subtraction	

Fig. 3.19 | Precedence of arithmetic operators.

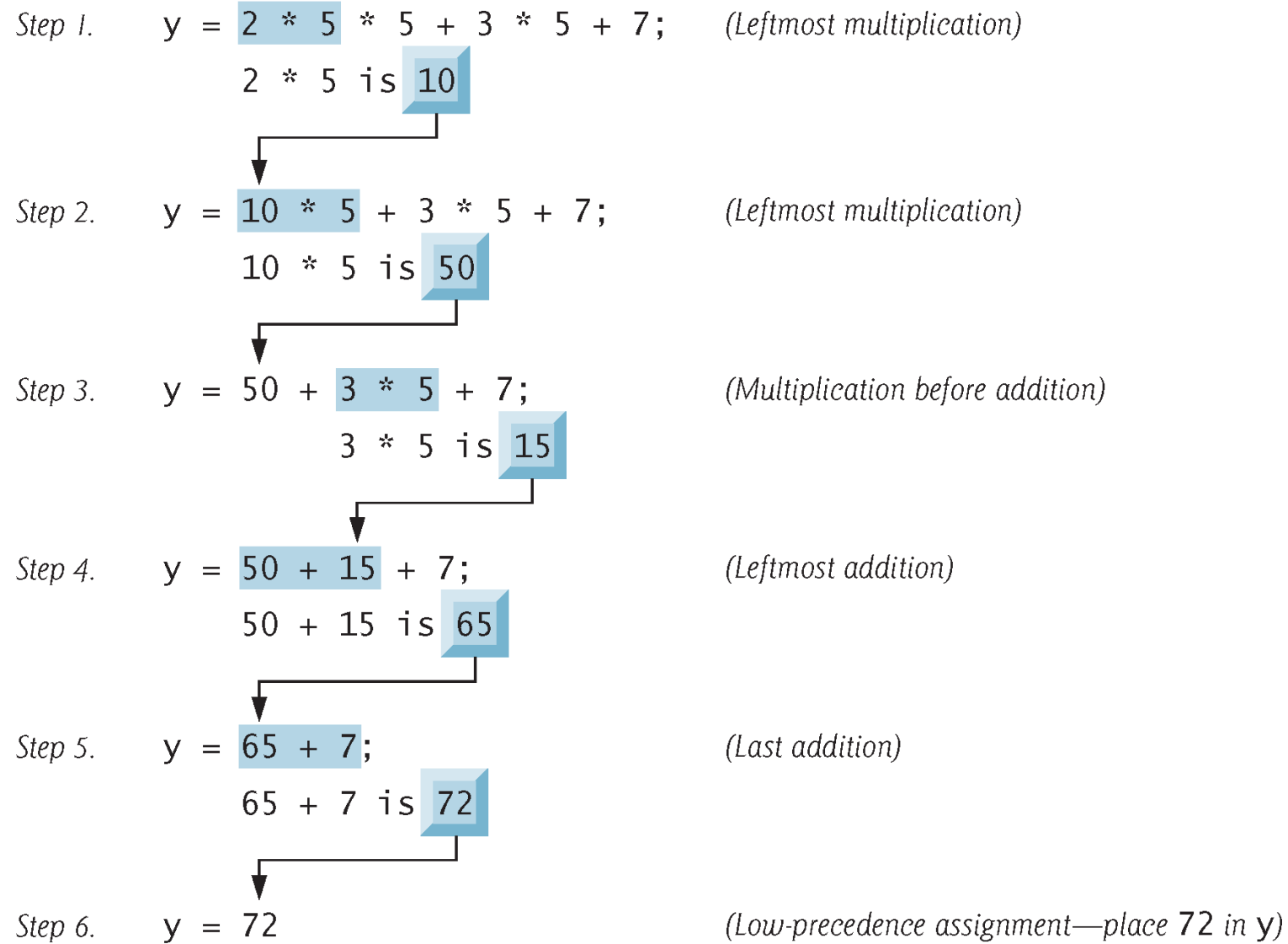


Fig. 3.20 | Order in which a second-degree polynomial is evaluated.

3.9 Decision Making: Equality and Relational Operators

- ▶ A **condition** is an expression that can be either **true** or **false**.
- ▶ Conditions can be formed using the **equality operators** (**==** and **!=**) and **relational operators** (**>**, **<**, **>=** and **<=**) summarized in Fig. 3.21.



Common Programming Error 3.5

Confusing the equality operator, `==`, with the assignment operator, `=`, can cause a logic error or a syntax error. The equality operator should be read as “is equal to,” and the assignment operator should be read as “gets” or “gets the value of.” To avoid confusion, some programmers read the equality operator as “double equals” or “equals equals.”

Standard algebraic equality and relational operators	C# equality or relational operator	Sample C# condition	Meaning of C# condition
<i>Relational operators</i>			
$>$	<code>></code>	<code>x > y</code>	x is greater than y
$<$	<code><</code>	<code>x < y</code>	x is less than y
$\geq$	<code>>=</code>	<code>x >= y</code>	x is greater than or equal to y
$\leq$	<code><=</code>	<code>x <= y</code>	x is less than or equal to y
<i>Equality operators</i>			
$=$	<code>==</code>	<code>x == y</code>	x is equal to y
$\neq$	<code>!=</code>	<code>x != y</code>	x is not equal to y

Fig. 3.21 | Relational and equality operators.

3.9 Decision Making: Equality and Relational Operators

- ▶ Figure 3.22 uses six **if statements** to compare two integers entered by the user.

```
1  // Fig. 3.22: Comparison.cs
2  // Comparing integers using if statements, equality operators
3  // and relational operators.
4  using System;
5
6  class Comparison
7  {
8      // Main method begins execution of C# app
9      static void Main()
10     {
11         // prompt user and read first number
12         Console.Write("Enter first integer: ");
13         int number1 = int.Parse(Console.ReadLine());
14
15         // prompt user and read second number
16         Console.Write("Enter second integer: ");
17         int number2 = int.Parse(Console.ReadLine());
18
19         if (number1 == number2)
20         {
21             Console.WriteLine($"{number1} == {number2}");
22         }
```

Fig. 3.22 | Comparing integers using if statements, equality operators and relational operators.
(Part I of 4.)

```
23
24     if (number1 != number2)
25     {
26         Console.WriteLine($"{number1} != {number2}");
27     }
28
29     if (number1 < number2)
30     {
31         Console.WriteLine($"{number1} < {number2}");
32     }
33
34     if (number1 > number2)
35     {
36         Console.WriteLine($"{number1} > {number2}");
37     }
38
39     if (number1 <= number2)
40     {
41         Console.WriteLine($"{number1} <= {number2}");
42     }
43
```

Fig. 3.22 | Comparing integers using if statements, equality operators and relational operators.
(Part 2 of 4.)

```
44         if (number1 >= number2)
45         {
46             Console.WriteLine($"{number1} >= {number2}");
47         }
48     } // end Main
49 } // end class Comparison
```

```
Enter first integer: 42
Enter second integer: 42
42 == 42
42 <= 42
42 >= 42
```

```
Enter first integer: 1000
Enter second integer: 2000
1000 != 2000
1000 < 2000
1000 <= 2000
```

Fig. 3.22 | Comparing integers using if statements, equality operators and relational operators.
(Part 3 of 4.)

```
Enter first integer: 2000
Enter second integer: 1000
2000 != 1000
2000 > 1000
2000 >= 1000
```

Fig. 3.22 | Comparing integers using if statements, equality operators and relational operators.
(Part 4 of 4.)

3.9 Decision Making: Equality and Relational Operators (Cont.)

- If the condition in an `if` statement is true, the statement associated with that `if` statement executes.
- An `if` statement always begins with keyword `if`, followed by a condition in parentheses.
- An `if` statement expects one statement in its body.



Good Programming Practice 3.10

Indent the statement(s) in the body of an if statement to enhance readability.



Error-Prevention Tip 3.6

You don't need to use braces, { }, around single-statement bodies, but you must include the braces around multiple-statement bodies. You'll see later that forgetting to enclose multiple-statement bodies in braces leads to errors. To avoid errors and make your code more readable, always enclose an `if` statement's body statement(s) in braces, even if it contains only a single statement.



Common Programming Error 3.6

Placing a semicolon immediately after the right parenthesis after the condition in an `if` statement is often a logic error (although not a syntax error). The semicolon causes the body of the `if` statement to be empty, so the `if` statement performs no action, regardless of whether or not its condition is true. Worse yet, the original body statement of the `if` statement now becomes a statement in sequence with the `if` statement and always executes, often causing the program to produce incorrect results.



Good Programming Practice 3.11

A lengthy statement can be spread over several lines. If a single statement must be split across lines, choose breaking points that make sense, such as after a comma in a comma-separated list, or after an operator in a lengthy expression. If a statement is split across two or more lines, indent all lines after the first until the end of the statement.

3.9 Decision Making: Equality and Relational Operators (Cont.)

- ▶ Figure 3.23 shows the precedence of the operators introduced in this chapter from top to bottom in decreasing order of precedence.



Good Programming Practice 3.12

Refer to the operator precedence chart (the complete chart is in Appendix A) when writing expressions containing many operators. Confirm that the operations in the expression are performed in the order you expect. If you're uncertain about the order of evaluation in a complex expression, use parentheses to force the order, as you would do in algebraic expressions. Some programmers also use parentheses to clarify the order. Observe that some operators, such as assignment, `=`, associate from right to left rather than left to right.

Operators				Associativity	Type
*	/	%		left to right	multiplicative
+	-			left to right	additive
<	<=	>	>=	left to right	relational
==	!=			left to right	equality
=				right to left	assignment

Fig. 3.23 | Precedence and associativity of operations discussed so far.